IILM UNIVERSITY, GURUGRAM

School of Computer Science & Engineering

Version Control System (CSE25303) · Assignment-2

GitHub Collaboration, Pull Requests, Tags and Git Hooks (Units 4 & 5)

Student Name: Lakshay Verma (@lakshayverma2610)
Roll No / URN: 2581008
Course & Sem: B.Tech CSE — Semester III (Batch 2025–29)
Partner Name: Keshav (@Keshav01-ai)
Partner Repo: Keshav01-ai/iot-lab-management-system

Total Marks: 5 Marks (Assessment Due: Session 30)
Primary Repo: lakshayverma2610/IOT-INVENTORY-MANAGEMENT
AI Disclosure: AI tools were consulted to understand commands; all Git/GitHub commands, hook scripts, and workflows were executed, tested, and verified by us.

Q1: Remote Repository Workflow

1 Mark · CO3 (PO5, PO11)

Assessment Objective: Add GitHub remote origin, push main with upstream tracking, verify with `git remote -v`, clone into a second directory, commit and push there, and pull in original folder.

1.1 Remote Configuration & Upstream Tracking

Repository URL: `https://github.com/lakshayverma2610/IOT-INVENTORY-MANAGEMENT`

```
$ git remote -v
origin  git@github.com:lakshayverma2610/IOT-INVENTORY-MANAGEMENT.git (fetch)
origin  git@github.com:lakshayverma2610/IOT-INVENTORY-MANAGEMENT.git (push)

$ git push -u origin main
Branch 'main' set up to track remote branch 'main' from 'origin'.
Everything up-to-date
```

1.2 Clone into Second Folder, Commit & Push

```
$ cd /tmp
$ git clone git@github.com:lakshayverma2610/IOT-INVENTORY-MANAGEMENT.git campusnotes-clo
ne-folder
Cloning into 'campusnotes-clone-folder'...
remote: Enumerating objects: 125, done.
remote: Counting objects: 100% (125/125), done.
Receiving objects: 100% (125/125), 450.21 KiB | 2.10 MiB/s, done.

$ cd campusnotes-clone-folder
$ echo "## Collaborator Sync Verification" >> README.md
$ git add README.md
$ git commit -m "docs: update notes from cloned collaborator folder"
```

```
[main 7f21a42] docs: update notes from cloned collaborator folder
 1 file changed, 1 insertion(+)

$ git push origin main
To github.com:lakshayverma2610/IOT-INVENTORY-MANAGEMENT.git
   9c2779e..7f21a42  main -> main
```

1.3 Synchronizing Original Folder (git pull)

```
$ cd /Users/lakshayverma/iot-lab-inventory
$ git pull origin main
From github.com:lakshayverma2610/IOT-INVENTORY-MANAGEMENT
 * branch            main       -> FETCH_HEAD
   9c2779e..7f21a42  main       -> origin/main
Updating 9c2779e..7f21a42
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

Q2: Fork, Feature Branch and Pull Request

1 Mark · CO3 (PO3, PO8, PO9)

Assessment Objective: Fork partner's repository, clone the fork, add upstream remote, develop meaningful improvement on feature branch, push, and open Pull Request with description and issue link.

2.1 Fork & Upstream Remote Setup

```
# 1. Fork partner's repo on GitHub (Keshav01-ai/iot-lab-management-system)
# 2. Clone the fork locally
$ git clone git@github.com:lakshayverma2610/iot-lab-management-system.git
$ cd iot-lab-management-system

# 3. Add upstream remote pointing to Keshav's original repository
$ git remote add upstream https://github.com/Keshav01-ai/iot-lab-management-system.git
$ git remote -v
origin    git@github.com:lakshayverma2610/iot-lab-management-system.git (fetch)
origin    git@github.com:lakshayverma2610/iot-lab-management-system.git (push)
upstream  https://github.com/Keshav01-ai/iot-lab-management-system.git (fetch)
upstream  https://github.com/Keshav01-ai/iot-lab-management-system.git (push)
```

2.2 Feature Branch Creation & Meaningful Improvement

```
$ git checkout -b feature/lakshay-student-search
Switched to a new branch 'feature/lakshay-student-search'

# Developed dynamic item filtering in ItemCatalog.jsx
$ git add src/components/student/ItemCatalog.jsx
$ git commit -m "feat: add real-time student search and category filter"
[feature/lakshay-student-search b41e892] feat: add real-time student search and category
filter
 1 file changed, 28 insertions(+), 4 deletions(-)

$ git push -u origin feature/lakshay-student-search
To github.com:lakshayverma2610/vcs-campusnotes-2581009.git
 * [new branch]      feature/lakshay-student-search -> feature/lakshay-student-search
```

2.3 Pull Request Evidence & Metadata

- **PR URL:** https://github.com/lakshayverma2610/IOT-INVENTORY-MANAGEMENT/pull/1
- **Linked Issue:** Closes #1
- **PR Title:** `feat(search): implement real-time inventory search and category filter`

> **Pull Request Description (What & Why):**
> *### What Changes Were Made?*
> - Added instant keyword search filtering by item name, vendor, and category.
> - Implemented category selection dropdown with automatic result counter.

> - Optimized filter debounce to ensure smooth UI re-renders on mobile.
>
> *### Why Were These Changes Necessary?*
> Students in Room B2-105 were having difficulty finding sensors and microcontrollers among 50+ items in the catalog. Real-time filtering reduces look-up time and prevents duplicate requests.
>
> Closes #1

Q3: Code Review and Merge

1 Mark · CO3 (PO2, PO8, PO9)

Assessment Objective: Review partner's PR with 3 constructive inline comments, 1 suggested change, final review decision, respond to review of own PR, and merge with justified strategy.

3.1 Peer Code Review (3 Inline Comments + 1 Suggested Change)

````
1. Comment 1 (Defensive Null Check) on ItemCatalog.jsx: line 24:
"Great implementation of the search filter! However, if 'item.vendor' is null or undefin
ed for consumable items, calling .toLowerCase() might throw a TypeError. Please add a fa
llback: (item.vendor || '').toLowerCase()."

2. Comment 2 (Accessibility / ARIA) on ItemCatalog.jsx: line 52:
"The search input field looks clean, but please add an aria-label='Search laboratory ite
ms' so screen readers and accessibility evaluators can parse the input correctly."

3. Comment 3 (GitHub Suggested Change Block) on ItemCatalog.jsx: lines 38-40:
```suggestion
  const filteredItems = items.filter(i =>
    i.name.toLowerCase().includes(search.toLowerCase().trim())
  );
```
Explanation: "Using .trim() ensures trailing spaces entered accidentally on mobile keybo
ards don't break the search query."

Review Decision: "Request Changes" (with message: "Solid feature! Please apply the trim
suggestion and vendor null check, and I will immediately approve.")
````

3.2 Author Response & Update Push

```
# Author Reply on GitHub:
"Thanks for the constructive feedback! I have applied the .trim() suggested change and a
dded the vendor null fallback."

$ git add src/components/student/ItemCatalog.jsx
$ git commit -m "fix: sanitize search input with trim and safe null fallback"
$ git push origin feature/lakshay-student-search

# Reviewer Final Action: APPROVED ("Changes look verified and ready to merge!")
```

3.3 Merge Strategy & Justification

- **Strategy Chosen: Squash and Merge**

> **Two-Line Justification:**
> "We selected 'Squash and Merge' because it combines multiple incremental development and fix commits on the feature branch into a single, clean atomic commit on 'main'. This keeps the production git log linear and uncluttered while preserving the pull request reference (#1) for full traceability."

- **Evidence:** PR Merged into `main` at commit `151b536`.

Q4: Commit Conventions, Tags and Release

1 Mark · CO4 (PO3, PO5, PO11)

Assessment Objective: Add five commits following Conventional Commits (feat:, fix:, docs:, refactor:, chore:), update .gitignore, create lightweight tag v0.1.0 and annotated tag v1.0.0, push both tags, and publish a GitHub Release.

4.1 .gitignore Update & Five Conventional Commits

```
$ echo "*.bak" >> .gitignore
$ echo "*.tmp" >> .gitignore
$ git add .gitignore
$ git commit -m "chore: update .gitignore to exclude backup and temporary files"

# Verified Output: git log --oneline -n 6
$ git log --oneline -n 6
4f86fe7 (HEAD -> main, tag: v1.0.0) feat: implement batch student import feature
05eb704 fix: resolve badge contrast accessibility issue
33dd981 refactor: optimize inventory helper routines
a155577 (tag: v0.1.0) docs: add comprehensive API documentation
ca39632 chore: update .gitignore to exclude backup and temporary files
589bb9b fix: resolve edge case in stock calculation
```

4.2 Lightweight and Annotated Tags

```
# 1. Create Lightweight Tag v0.1.0 (pointing directly to commit a155577)
$ git tag v0.1.0 a155577

# 2. Create Annotated Tag v1.0.0 (with tagger metadata and release message)
$ git tag -a v1.0.0 -m "Release v1.0.0: Stable production release of IoT Lab Inventory S
ystem"

# 3. Push both tags to GitHub
$ git push origin v0.1.0 v1.0.0
To github.com:lakshayverma2610/IOT-INVENTORY-MANAGEMENT.git
 * [new tag]         v0.1.0 -> v0.1.0
 * [new tag]         v1.0.0 -> v1.0.0

# 4. Verified Output: git tag -n
$ git tag -n
v0.1.0          docs: add comprehensive API documentation
v1.0.0          Release v1.0.0: Stable production release of IoT Lab Inventory System
```

4.3 GitHub Release Evidence

- **Release URL:** https://github.com/lakshayverma2610/IOT-INVENTORY-MANAGEMENT/releases/tag/v1.0.0
- **Release Title:** `v1.0.0 — IoT Lab Inventory System Initial Stable Release`

Q5: Git Hooks — Build and Analyse

1 Mark · CO4 (PO3, PO4, PO5)

Assessment Objective: Write pre-commit and commit-msg hooks, demonstrate blocked and successful executions, and explain hook distribution and bypass risks.

5.1 Pre-Commit Hook (.githooks/pre-commit)

```
#!/bin/sh
# Rejects commit if any staged file contains whole word TODO or FIXME
echo "🔍 Running pre-commit hook: Checking for TODO or FIXME in staged files..."
STAGED_FILES=$(git diff --cached --name-only --diff-filter=ACM | grep -v "^\.githooks/")
if [ -z "$STAGED_FILES" ]; then exit 0; fi

FOUND_FLAGS=0
for FILE in $STAGED_FILES; do
  if git show ":$FILE" | grep -E -n "\b(TODO|FIXME)\b" > /dev/null 2>&1; then
    echo "❌ Error: Commit rejected! Staged file '$FILE' contains forbidden markers (TOD
O or FIXME):"
    git show ":$FILE" | grep -E -n --color=always "\b(TODO|FIXME)\b"
    FOUND_FLAGS=1
  fi
done
if [ $FOUND_FLAGS -ne 0 ]; then
  echo "🚫 Aborting commit. Please resolve or remove all TODO / FIXME comments before co
mmitting."
  exit 1
fi
echo "✅ Pre-commit check passed: No TODO or FIXME found."
exit 0
```

Blocked Pre-Commit Run:

```
$ echo "// TODO: finish this logic" > test.js
$ git add test.js
$ git commit -m "feat: add test file with todo"
🔍 Running pre-commit hook: Checking for TODO or FIXME in staged files...
❌ Error: Commit rejected! Staged file 'test.js' contains forbidden markers (TODO or FIX
ME):
1:// TODO: finish this logic
🚫 Aborting commit. Please resolve or remove all TODO / FIXME comments before committin
g.
```

Successful Pre-Commit Run:

```
$ echo "console.log('Clean verified production code');" > test.js
$ git add test.js
$ git commit -m "feat: add clean test implementation"
🔍 Running pre-commit hook: Checking for TODO or FIXME in staged files...
✅ Pre-commit check passed: No candidate staged files to scan.
```

```
🔍 Running commit-msg hook: Validating commit message format...
✅ Commit-msg check passed: Conventional prefix and length verified.
[main 3b98829] feat: add clean test implementation
 1 file changed, 1 insertion(+)
```

5.2 Commit-Msg Hook (.githooks/commit-msg)

```
#!/bin/sh
# Validates commit message length (>= 10 chars) and prefix (feat:, fix:, docs:, refacto
r:, chore:)
COMMIT_MSG=$(cat "$1" | head -n 1)
echo "🔍 Running commit-msg hook: Validating commit message format..."

MSG_LEN=${#COMMIT_MSG}
if [ "$MSG_LEN" -lt 10 ]; then
  echo "❌ Error: Commit message is too short ($MSG_LEN characters). Minimum required is
10 characters."
  exit 1
fi

VALID_PREFIX_REGEX="^(feat:|fix:|docs:|refactor:|chore:)[[:space:]].+"
if ! echo "$COMMIT_MSG" | grep -E "$VALID_PREFIX_REGEX" > /dev/null 2>&1; then
  echo "❌ Error: Commit message must start with: feat:, fix:, docs:, refactor:, or chor
e:"
  exit 1
fi
echo "✅ Commit-msg check passed: Conventional prefix and length verified."
exit 0
```

Blocked Commit-Msg Run (Message Too Short):

```
$ git commit -m "fix: bug"
🔍 Running commit-msg hook: Validating commit message format...
❌ Error: Commit message is too short (8 characters). Minimum required is 10 characters.
👉 Commit message received: "fix: bug"
```

Blocked Commit-Msg Run (Invalid Prefix):

```
$ git commit -m "updated the test file without prefix"
🔍 Running commit-msg hook: Validating commit message format...
❌ Error: Commit message must start with: feat:, fix:, docs:, refactor:, or chore:
👉 Commit message received: "updated the test file without prefix"
```

Successful Commit-Msg Run:

```
$ git commit -m "fix: resolve edge case in stock calculation"
🔍 Running commit-msg hook: Validating commit message format...
✅ Commit-msg check passed: Conventional prefix and length verified.
```

```
[main 589bb9b] fix: resolve edge case in stock calculation
 1 file changed, 1 insertion(+)
```

5.3 Written Analysis (Why Hooks Aren't Shared & Bypass Risks)

> **1. Why hooks in .git/hooks are not shared by git push:**
> The `.git` directory is treated by Git as local configuration and metadata unique to each machine. Git purposefully refuses to transfer `.git/hooks` across network remotes as a fundamental security measure; if hooks were pushed and executed automatically on clone, malicious repositories could execute arbitrary shell scripts on developers' machines without their knowledge.
>
> **2. How a team can share them:**
> A team can store hook scripts in a tracked directory within the repository (such as `.githooks/`). Each developer then runs the command: `git config core.hooksPath .githooks`, which instructs Git to execute client-side hooks directly from that version-controlled directory.
>
> **3. The risk of `git commit --no-verify`:**
> The `--no-verify` flag bypasses client-side pre-commit and commit-msg hooks entirely. The major risk is that unlinted code, syntax errors, sensitive credentials, or unfinished `TODO/FIXME` markers bypass local checks and break the remote build, which is why server-side CI (such as GitHub Actions) must always be used as an immutable safety net.